

Rapport de Projet: Application Bibliotheca

Projet Bibliotheca – Application Web de Gestion de Bibliothèque

Rapport de Projet Front-End

Smart Backoffice Dashboard

Réalisé par:
HAYMER Salma
SAAD Ibtissam

Encadré par :
Tlemçani khadija

Filière : 3ème année Ingénieur en Informatique
Module : Développement Web
Année universitaire : 2025 / 2026

Table des matières:

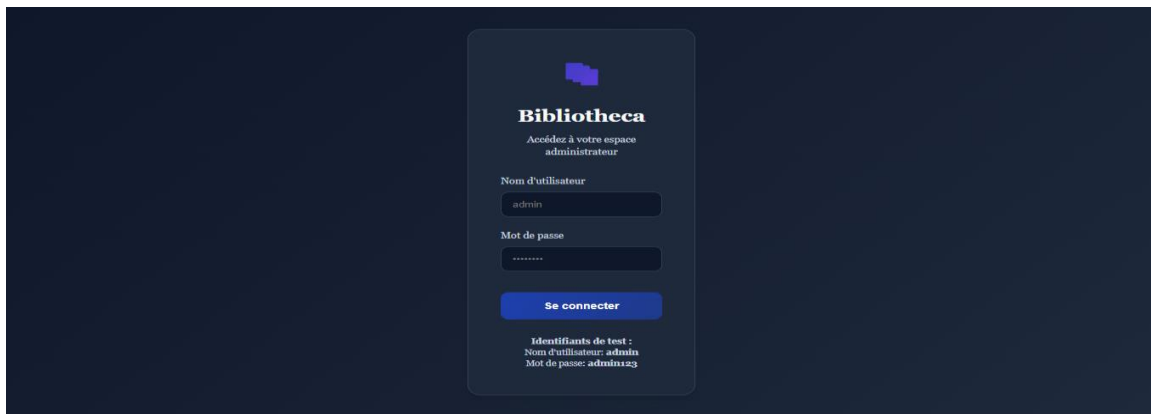
- 1) Introduction générale
- 2) Analyse du sujet et compréhension des besoins
- 3) Conception générale de l'application
- 4) Structure SPA et navigation
- 5) Développement du Module 1 – CRUD complet (Livres)
- 6) Développement du Module 2 – CRUD simple (Auteurs)
- 7) Dashboard et indicateurs KPI
- 8) Intégration de l'API externe
- 9) Tests et validation
- 10) Difficultés et choix techniques
- 11) Conclusion et perspectives

1. INTRODUCTION GÉNÉRALE:

Ce projet s'inscrit dans le cadre du module de Développement Web et a pour objectif la réalisation d'une application web de type Backoffice Dashboard. Le sujet choisi, Bibliotheca, est un système de gestion de bibliothèque permettant l'administration complète d'une collection de livres.

L'application a été développée exclusivement avec les technologies web fondamentales : HTML5, CSS3 et JavaScript Vanilla, conformément aux contraintes pédagogiques qui interdisent l'utilisation de frameworks. Cette contrainte nous a permis de maîtriser les manipulations avancées du DOM, la gestion des événements et l'organisation modulaire d'une application.

Bibliotheca répond aux exigences d'une Single Page Application (SPA) avec une interface moderne et intuitive, comprenant un tableau de bord analytique, un système CRUD complet pour la gestion des livres, un module secondaire pour les auteurs, et une intégration d'API externe pour enrichir le catalogue.



2. ANALYSE DU SUJET ET COMPRÉHENSION DES BESOINS:

a) Contexte et exigences:

Le cahier des charges imposait le développement d'un dashboard backoffice avec trois modules obligatoires :

- Un module principal avec CRUD complet
- Un module secondaire avec CRUD simplifié
- Un module dashboard avec KPI et intégration API

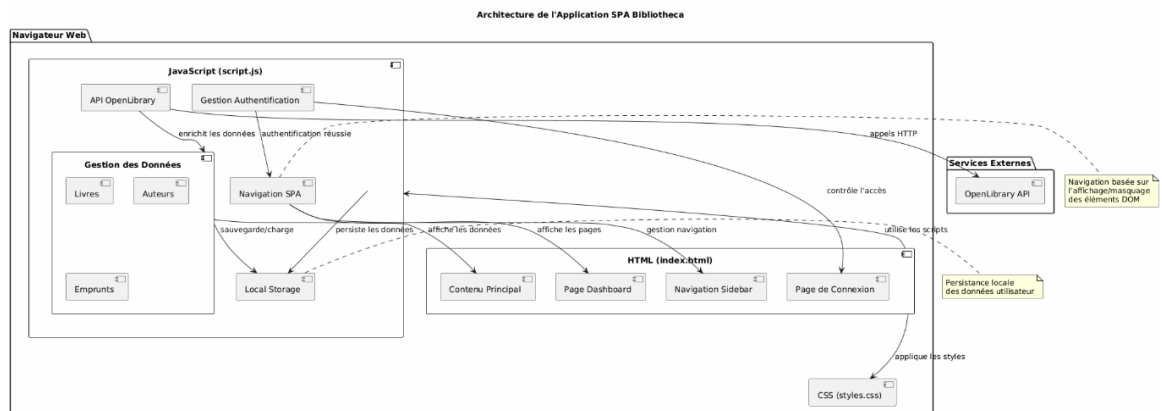
Parmi les cinq sujets proposés, nous avons choisi Bibliotheca qui correspond au thème Livres/Auteurs/OpenLibrary. Ce choix nous a permis de travailler sur un domaine familier tout en répondant à toutes les exigences techniques.

b) Besoins fonctionnels identifiés:

- Gestion des livres : Ajout, consultation, modification, suppression
- Gestion des auteurs : Création et suppression
- Tableau de bord : Statistiques en temps reel
- Persistance : Sauvegarde des données localement
- Interface : Navigation intuitive et responsive

c) Contraintes techniques:

- Application monopage (SPA)
- Sans frameworks JavaScript
- Utilisation de LocalStorage
- Intégration d'API REST externe
- Responsive design



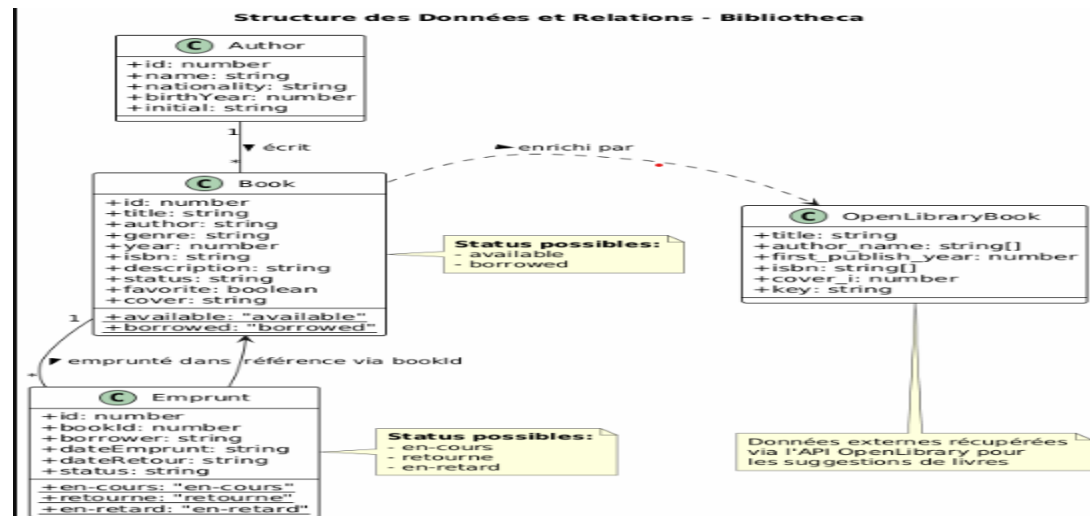
3) CONCEPTION GÉNÉRALE DE L'APPLICATION:

a. Architecture technique:

- L'application suit une architecture modulaire organisée autour des composants suivants :
- Couche présentation : HTML/CSS pour l'interface
- Couche logique : JavaScript pour la gestion des données
- Couche persistance : LocalStorage pour le stockage
- Couche externe : API OpenLibrary pour l'enrichissement

b. Structure des données:

Les données sont structurées en quatre collections principales :



c. Organisation des fichiers:

 salma-hy Update index.html ✓	
 README.md	Update README.md
 index.html	Update index.html
 index.login	Update index.login
 script.js	Update script.js
 styles.css	Update styles.css

4. STRUCTURE SPA ET NAVIGATION:

a. Principe de la Single Page Application:

- L'application implémente le pattern SPA en masquant/affichant des sections de contenu plutôt qu'en rechargeant des pages. Tous les contenus sont présents dans un seul fichier HTML (index.html) et la navigation est gérée via JavaScript.

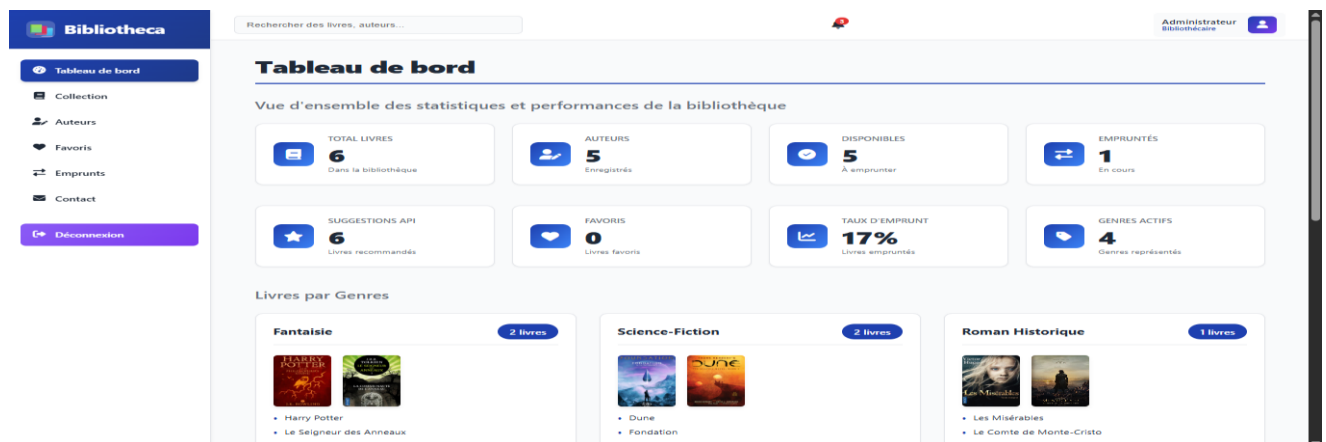
b. Système de navigation:

- La navigation s'effectue via une sidebar fixe contenant les liens vers les différentes sections :
 - Tableau de bord
 - Collection (livres)
 - Auteurs
 - Favoris
 - Emprunts
 - Contact
- Chaque clic sur un lien déclenche :
 - La désactivation de la section courante
 - L'activation de la section cible
 - La mise à jour du lien actif dans la sidebar

c. Header et recherche globale:

- Un header fixe en haut de l'application contient :
 - Une barre de recherche globale
 - Les notifications
 - Le profil utilisateur

La recherche globale permet de filtrer instantanément le contenu de la section active.



5. DÉVELOPPEMENT DU MODULE 1 – CRUD COMPLET (LIVRES) :

a. Create - Ajout d'un livre :

L'ajout d'un livre s'effectue via un modal contenant un formulaire complet. Chaque champ est validé côté client avant soumission.

Fonctionnalités implémentées :

- Validation des champs requis
- Sélection du genre via liste déroulante
- Vérification de l'année de publication
- Upload d'URL de couverture

b. Read - Consultation des livres

L'affichage des livres se fait sous forme de cartes avec :

- Image de couverture
- Titre et auteur
- Genre et année
- Statut de disponibilité
- Boutons d'action

Fonctionnalités :

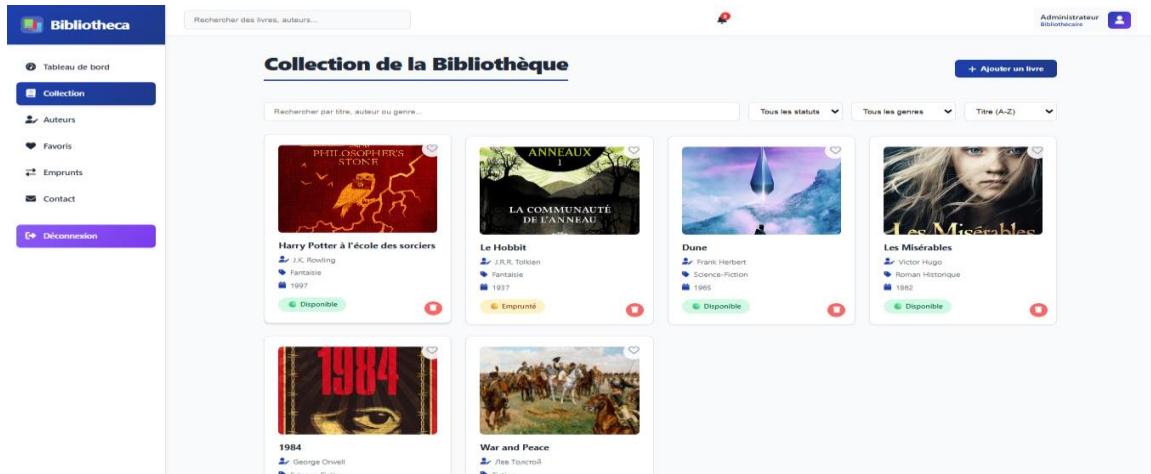
- Affichage en grille responsive
- Fiche détaillée au clic
- Filtrage par statut et genre
- Recherche par mot-clé
- Tri par titre, année, auteur

c. Update - Modification d'un livre

La modification utilise le même modal que l'ajout, pré-rempli avec les données du livre sélectionné.

d. Delete - Suppression d'un livre

La suppression est protégée par une confirmation et supprime également les emprunts associés.



6. DÉVELOPPEMENT DU MODULE 2 – CRUD SIMPLE (AUTEURS)

a. Structure simplifiée

Contrairement au module livres, le module auteurs implémente un CRUD light avec :

- Ajout via formulaire simple
- Affichage sous forme de cartes
- Suppression avec vérification

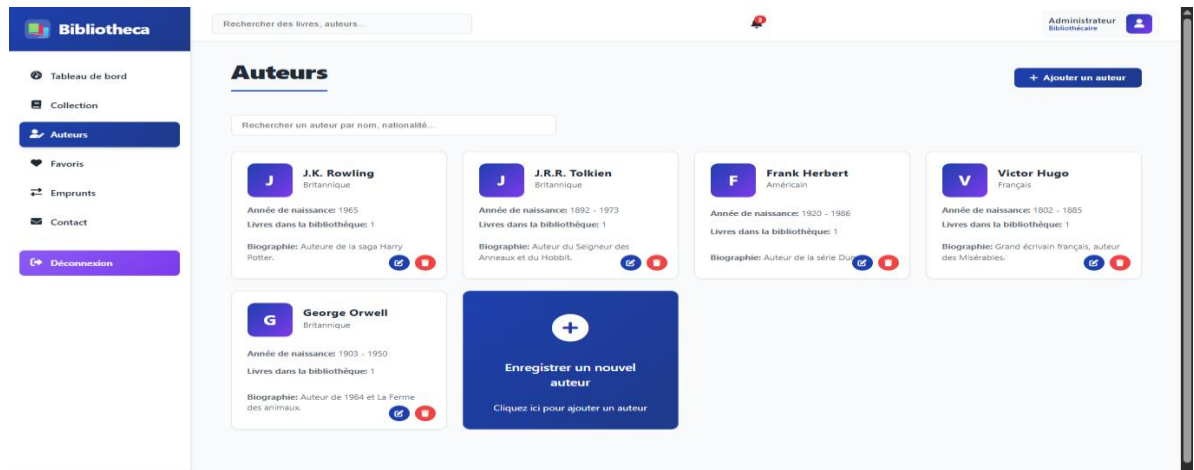
b. Liaison avec le Module 1

Les auteurs sont liés aux livres via le champ author. La suppression d'un auteur déclenche une vérification des livres associés et propose de les supprimer également.

c. Interface utilisateur

Les auteurs sont affichés dans une grille avec :

- Initiale dans un avatar coloré
- Nom et nationalité
- Nombre de livres dans la bibliothèque
- Boutons d'édition et suppression



7. DASHBOARD ET INDICATEURS KPI

a. Conception du tableau de bord

Le dashboard offre une vue synthétique de l'état de la bibliothèque avec huit indicateurs KPI :

1. **Total livres** : Nombre total de livres
2. **Auteurs** : Nombre d'auteurs enregistrés
3. **Disponibles** : Livres disponibles à l'emprunt
4. **Empruntés** : Livres actuellement empruntés
5. **Suggestions API** : Livres recommandés
6. **Favoris** : Livres marqués comme favoris
7. **Taux d'emprunt** : Pourcentage de livres empruntés
8. **Genres actifs** : Nombre de genres représentés

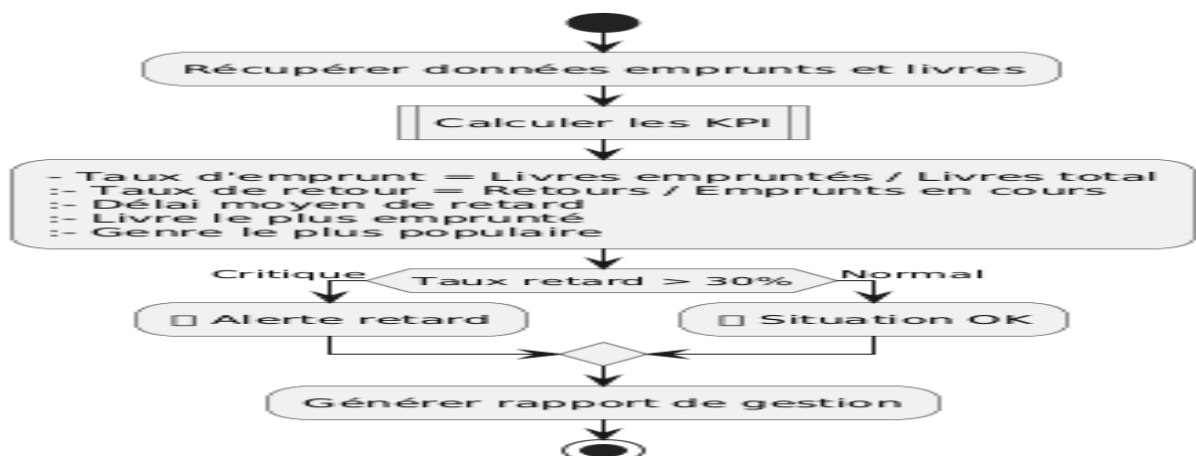
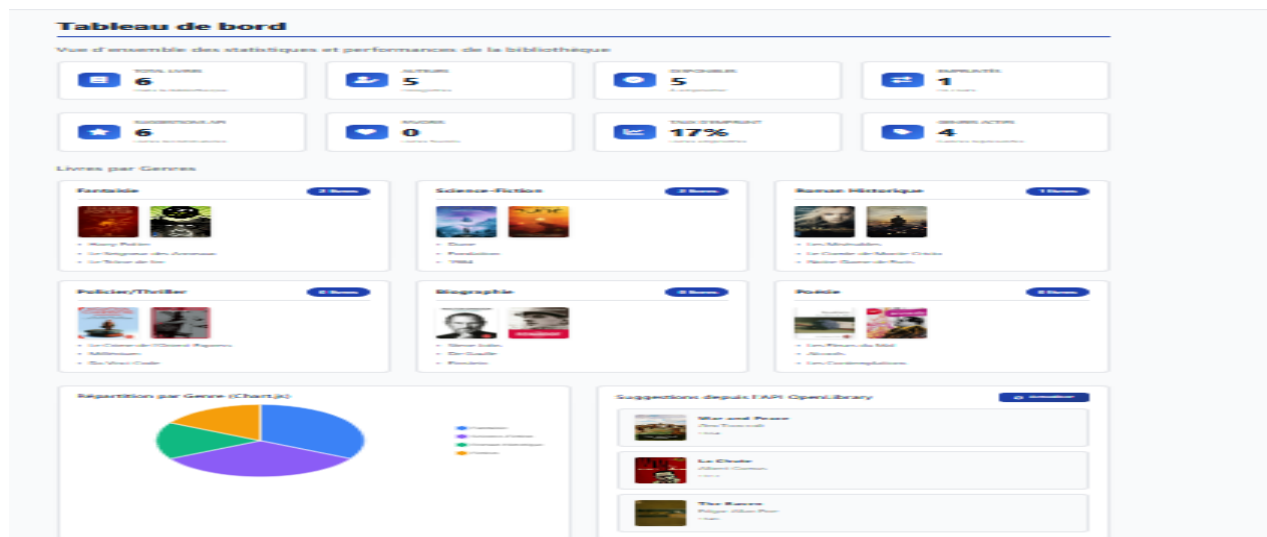
b. Calcul des indicateurs

Chaque KPI est calculé en temps réel à partir des données :

```
function updateDashboardStats() {  
    document.getElementById("total-books").textContent = books.length;  
    document.getElementById("total-authors").textContent = authors.length;  
}
```

c. Visualisation avec Chart.js

Un graphique circulaire (pie chart) montre la répartition des livres par genre. Le graphique est interactif et met à jour sa légende au survol.



8. INTÉGRATION DE L'API EXTERNE

a. Choix de l'API OpenLibrary

Nous avons sélectionné l'API OpenLibrary pour sa gratuité, sa documentation complète et la richesse de ses données bibliographiques.

b. Implémentation avec fetch()

L'intégration utilise l'API fetch() avec gestion des promesses :

```
async function loadSuggestions() {
  const container = document.getElementById("suggestions-list");
  if (!container) return;

  container.innerHTML = "Chargement...";

  try {
    const res = await fetch(
      "https://openlibrary.org/search.json?q=classic+literature&limit=6"
    );
    const data = await res.json();

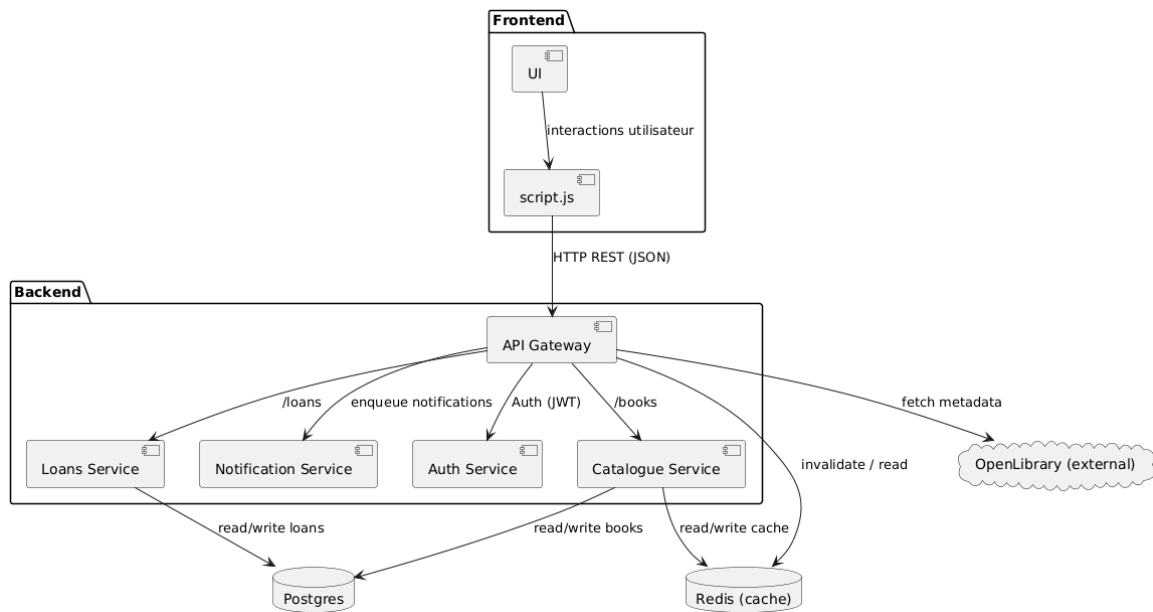
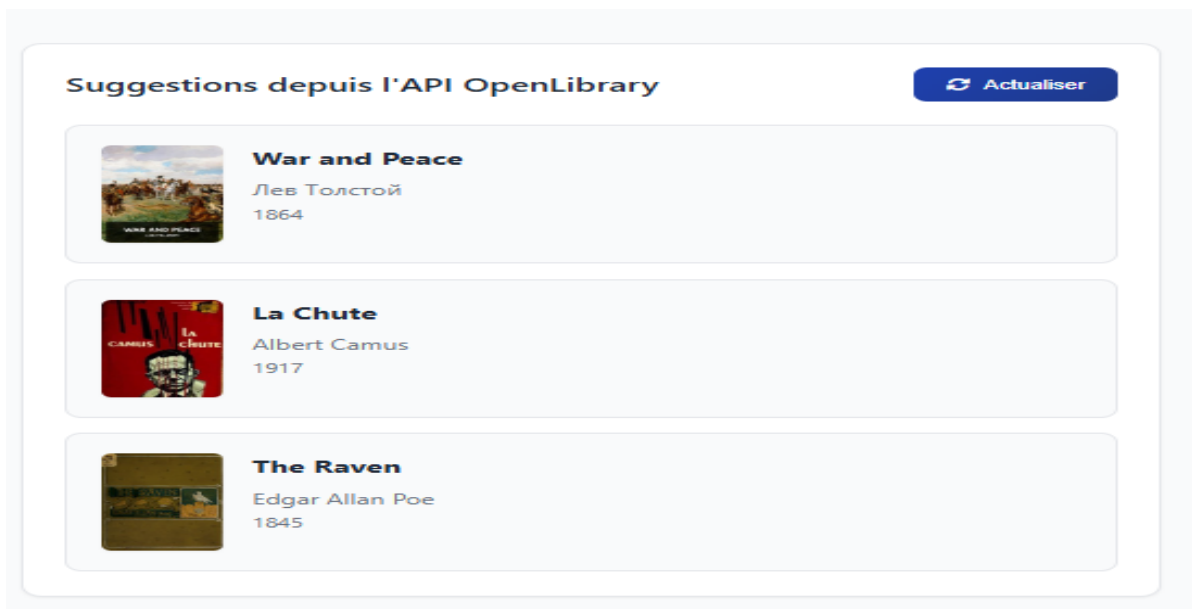
    container.innerHTML = "";
  }
}
```

c. Fonctionnalités implémentées

- Récupération de livres populaires
- Affichage des suggestions avec couvertures
- Ajout automatique d'un livre de l'API à la collection
- Gestion des erreurs (timeout, réseau, format)
- Bouton de rafraîchissement manuel

d. Impact sur les KPI

L'intégration API met à jour le KPI "Suggestions API" et enrichit occasionnellement la base locale avec de nouveaux livres.



9. TESTS ET VALIDATION

a. Méthodologie de test

Nous avons adopté une approche de test manuel systématique pour chaque fonctionnalité

Fonctionnalité	Test effectué	Résultat
Connexion	Identifiants valides/invalides	☐

Ajout livre	Formulaire complet/partiel	☐
Modification	Mise à jour des champs	☐
Suppression	Confirmation et cascade	☐
Recherche	Mot-clé présent/absent	☐
Tri	Par titre, année, auteur	☐
Favoris	Ajout/retrait	☐
Emprunts	Création et retour	☐
Persistence	Rechargement page	☐
API	Connexion et erreurs	☐

b. Tests de persistance

La persistance via LocalStorage a été testée en :

1. Ajoutant des données
2. Rechargeant la page (F5)
3. Vérifiant la présence des données

c. Tests responsive

L'interface a été testée sur trois viewports :

- **Desktop** ($\geq 1024\text{px}$) : Interface complète
- **Tablette** (768px) : Adaptation des grilles
- **Mobile** ($< 480\text{px}$) : Menu hamburger implicite

10. DIFFICULTÉS ET CHOIX TECHNIQUES

a. Difficulté 1 : Synchronisation LocalStorage ↔ DOM

Problème : Maintenir la cohérence entre les données en mémoire et l'affichage.

Solution : Implémentation de deux fonctions centrales :

- `saveAllData()` : Sérialise et sauvegarde tout
- `loadAllData()` : Charge et désérialise

Choix technique : Un seul point de sauvegarde pour garantir la cohérence.

b. Difficulté 2 : Gestion asynchrone de l'API

Problème : Gérer les états de chargement, succès et erreur.

Solution : Utilisation d'async/await avec try/catch et interface d'attente.

Choix technique : Fallback visuel en cas d'erreur plutôt que blocage.

c. Difficulté 3 : Navigation SPA sans rechargement

Problème : Changer de section sans recharger la page.

Solution : Système de classes CSS active/inactive géré par JavaScript.

Choix technique : Tous les contenus chargés initialement, masqués/affichés dynamiquement.

d. Autres choix techniques

- **Chart.js** plutôt que D3.js pour sa simplicité
- **CSS Grid** pour les layouts complexes
- **Flexbox** pour l'alignement
- **Variables CSS** pour le thème
- **Font Awesome** pour les icônes

11. CONCLUSION ET PERSPECTIVES

a. Bilan du projet

Le projet **Bibliotheca** a permis d'atteindre tous les objectifs pédagogiques fixés :

- ✓ Maîtrise de la manipulation avancée du DOM
- ✓ Gestion des événements utilisateur
- ✓ Organisation modulaire d'une application
- ✓ Implémentation de fonctionnalités CRUD
- ✓ Persistance des données avec LocalStorage
- ✓ Création d'un dashboard professionnel
- ✓ Intégration d'API externe asynchrone

b. Acquis pédagogiques

Ce projet constitue une préparation solide aux frameworks JavaScript modernes en ayant abordé :

- L'architecture d'une SPA
- La gestion d'état
- Les appels API asynchrones
- La persistance client-side
- Le responsive design

c. Améliorations possibles

Pour une version future de l'application, nous envisageons :

1. **Authentification avancée** : Multi-utilisateurs avec rôles
2. **Export de données** : PDF des statistiques, CSV du catalogue
3. **Système de réservation** : File d'attente pour livres empruntés
4. **Notifications** : Email pour retards de retour
5. **Recherche avancée** : Filtres combinés, recherche plein texte
6. **Mode hors-ligne** : Service Worker pour PWA
7. **Internationalisation** : Support multi-langues

d. Conclusion finale

Bibliotheca démontre qu'il est possible de créer une application web complète et professionnelle en utilisant uniquement les technologies web natives. Le projet répond intégralement au cahier des charges tout en offrant une expérience utilisateur moderne et intuitive.

Le code source est disponible sur GitHub : <https://github.com/salma-hy/bibliotheca-dashboard>

○ ANNEXES

Annexe A : Références techniques

- Documentation Chart.js : <https://www.chartjs.org/>
- API OpenLibrary : <https://openlibrary.org/developers/api>
- MDN Web Docs : <https://developer.mozilla.org/fr/>

Annexe B : Manuel d'utilisation rapide

1. Se connecter avec : admin / admin123
2. Consulter le tableau de bord pour les statistiques
3. Ajouter des livres via "Collection" → "Ajouter un livre"
4. Gérer les auteurs dans la section dédiée
5. Marquer des favoris avec l'icône cœur
6. Gérer les emprunts dans la section correspond